Subject: ASD&D	**Semester:** V
Class: TE	**Batch:** T11
Name: Ananya Amesur	**Roll No:** 2401005

Experiment 8

Aim: To implement CI/CD pipeline to host a website on AWS

Lab Outcome: LO3

LO3: Build and manage automated CI/CD pipelines for application development, testing, packaging, and deployment.

Theory:

Continuous Integration and Continuous Deployment (CI/CD)

Continuous Integration (CI) is a software development practice where developers merge their code changes frequently into a central shared repository, followed by automated builds and automated test executions to detect integration bugs early. Continuous Delivery/Deployment (CD) extends this practice by automating the release process so that verified code changes can be automatically packaged and deployed directly to staging or production environments without manual intervention.

Implementing CI/CD drastically shortens the software development lifecycle (SDLC), minimizes manual deployment errors, improves software stability, and enables rapid iteration of software products.

Key Components of AWS CI/CD Architecture

- Source Repository (GitHub): Acts as the centralized version control system (VCS). Webhooks or OAuth connection points alert the cloud pipeline instantaneously whenever commits are pushed to specific branches (e.g., main).
- AWS CodePipeline: A continuous delivery orchestration service that models, visualizes, and coordinates every step of the release process. It manages the

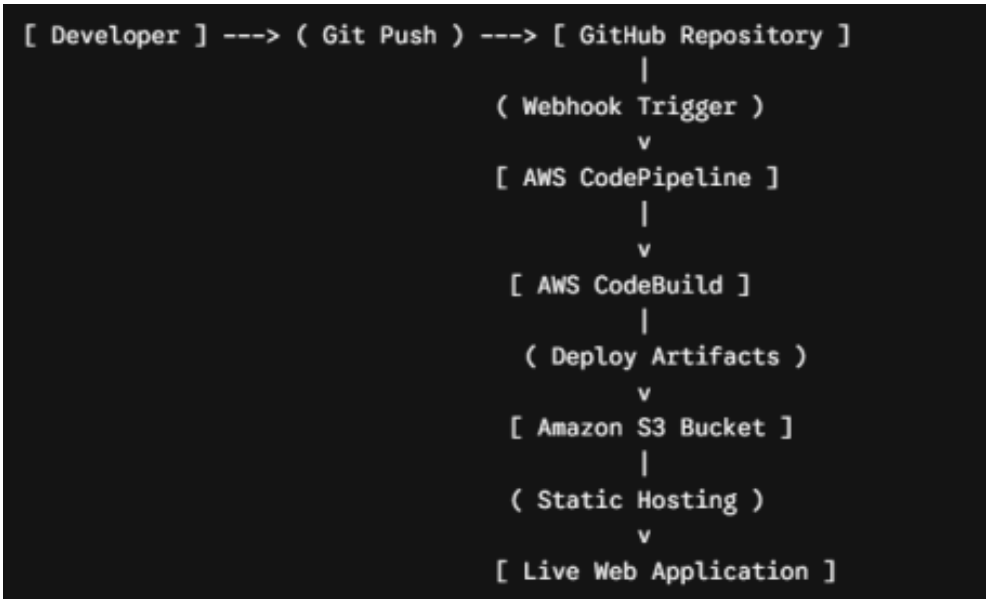
stages—such as Source, Build, and Deploy—moving build artifacts sequentially through execution pipelines.

- AWS CodeBuild: A fully managed, scalable build and compilation service. CodeBuild spins up isolated ephemeral compute containers, runs configuration scripts specified in a buildspec.yml file, runs unit tests, compiles static assets, and bundles output artifacts for downstream deployment.
- Amazon Simple Storage Service (Amazon S3): Provides scalable object storage with built-in Static Website Hosting. When configured with public read access (s3:GetObject) and an index.html root document, it serves as a lightweight, highly durable, serverless web server directly from bucket storage.
- AWS Identity and Access Management (IAM): Enforces fine-grained least-privilege security. Service roles ensure CodeBuild and CodePipeline have the exact permissions required (such as read/write access to S3 artifact and deployment buckets) to operate securely without exposing root or persistent user credentials.

Operational Workflow

- Push: A developer commits changes to the GitHub repository.
- Trigger: GitHub triggers an automated event notification to AWS CodePipeline via webhook integration.
- Build: CodePipeline retrieves the source files and triggers AWS CodeBuild to execute containerized compilation and packaging.
- Deploy: The pipeline extracts the verified build artifacts directly into the designated S3 deployment bucket.
- Serve: The Amazon S3 static website hosting endpoint renders the live web application immediately to users.

Architecture & Workflow:

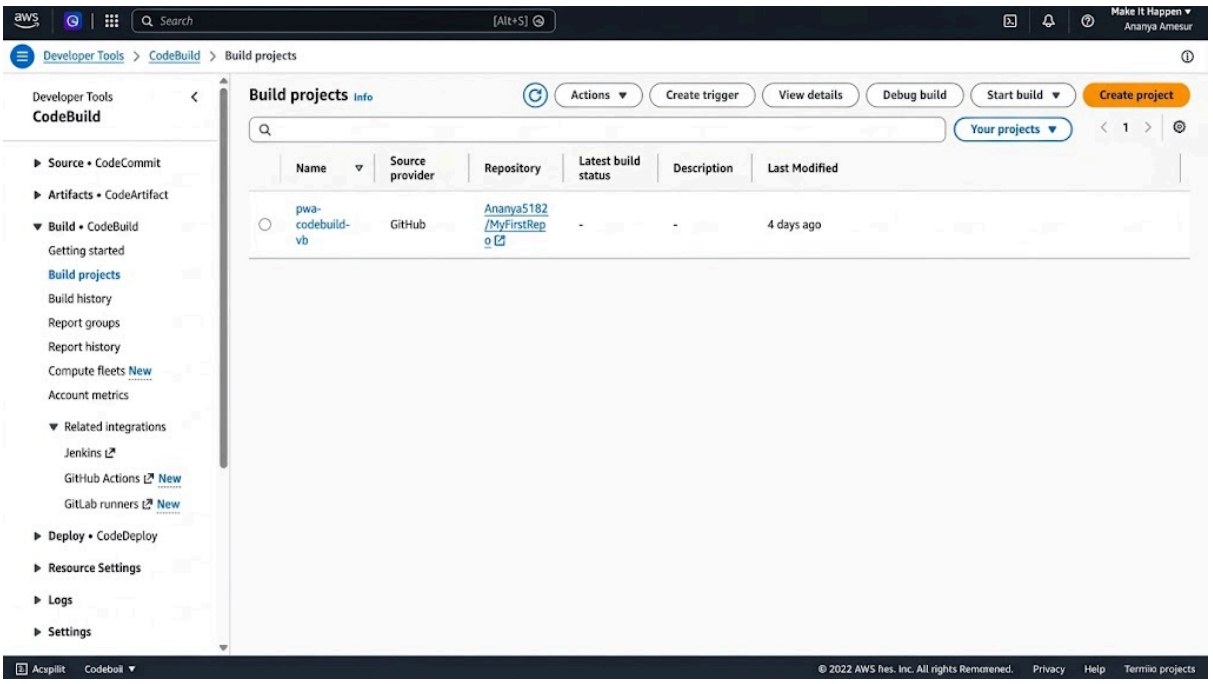


Implementation:

Phase 1: AWS CodeBuild Project Configuration

Step 1: Create a Build Project

1. Open the AWS Management Console and navigate to Developer Tools > CodeBuild.
2. Click on Create build project.
3. Under Project configuration, set the Project name to pwa-codebuild-vb.
4. Select Default project as the project type.

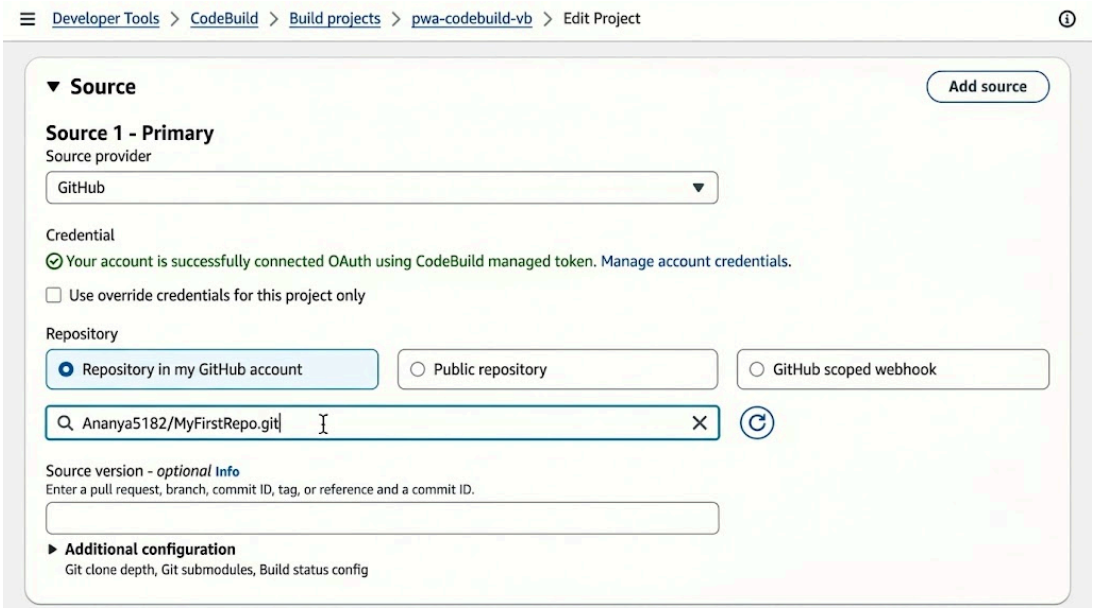


Step 2: Configure Source & Authenticate GitHub

1. Under the Source section, select GitHub as the Source provider.

2. Select OAuth app as the credential type.

3. Click on Connect to GitHub and complete the OAuth authorization prompt (Authorize aws-codesuite) to link your GitHub account.



Step 3: Repository Selection & Environment Setup

1. Choose Repository in my GitHub account.

2. Select the repository URL.

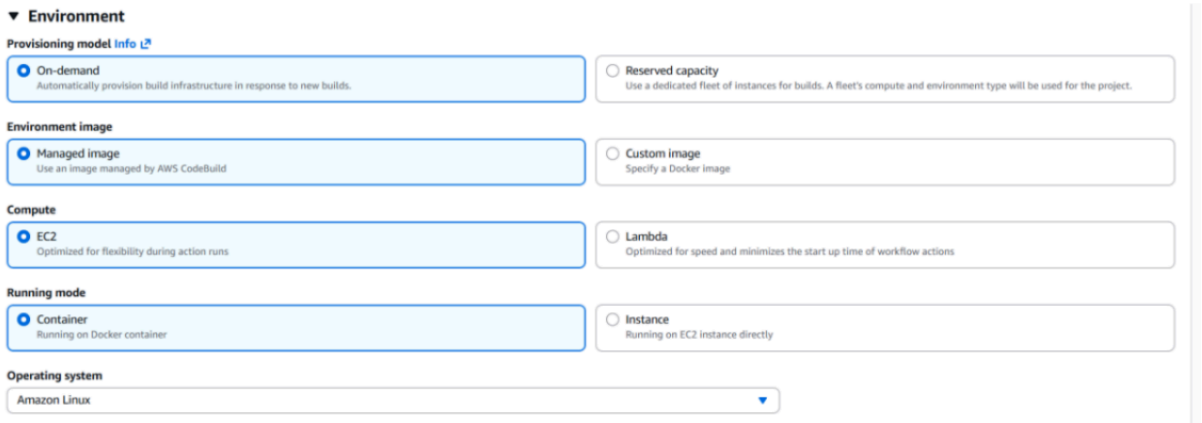
3. Scroll down to Environment:

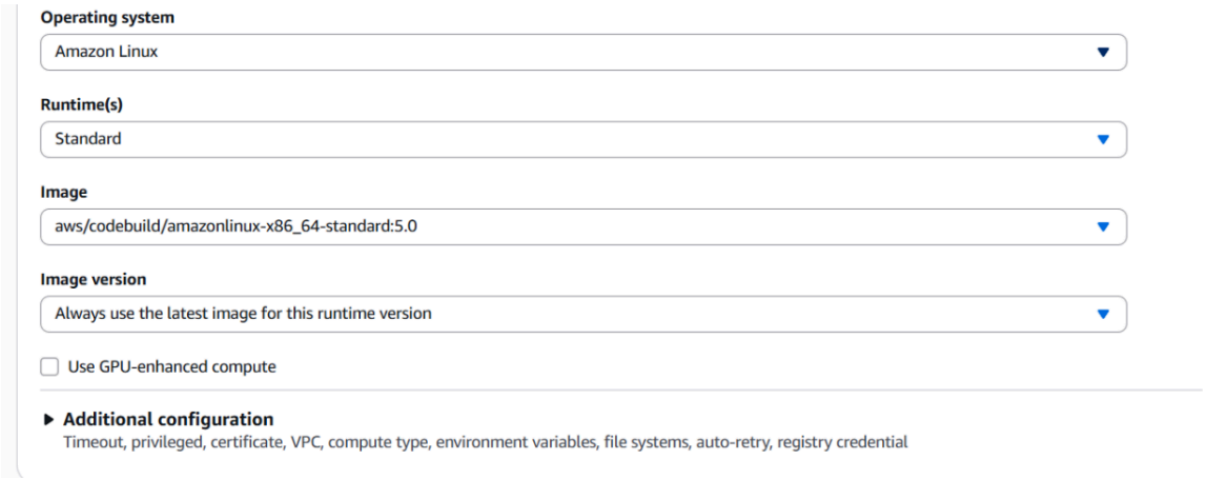
- o **Provisioning model:** On-demand
- o **Environment image:** Managed image
- o **Compute:** EC2
- o **Running mode:** Instance
- o **Operating system:** Amazon Linux
- o **Runtime(s):** Base

4. Under Service role, choose New service role and verify the generated role name (codebuild-pwa-codebuild-vb-service-role).

5. Under Buildspec, select Use a buildspec file (defaulting to buildspec.yml in the root repository).

6. Under Artifacts, select No artifacts (as CodePipeline will manage artifacts). Enable CloudWatch logs and click Create build project.

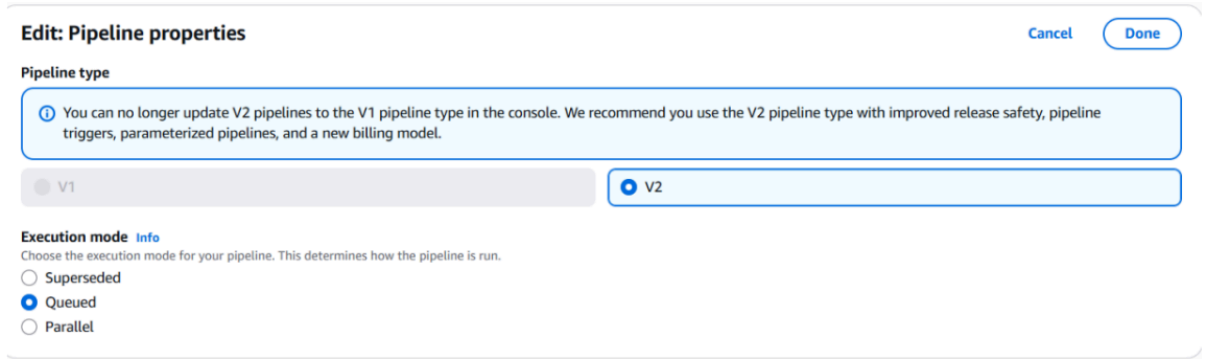




Phase 2: AWS CodePipeline Setup

Step 4: Pipeline Settings & Artifact Storage

1. Navigate to Developer Tools > CodePipeline and click Create pipeline.
2. Enter the pipeline name as pwa-pipeline.
3. Set Execution mode to Queued and select New service role.
4. Expand Advanced settings:
    - o Artifact store: Custom location
    - o Bucket: Select your S3 artifact bucket (e.g., store-build-artifact-vb)
    - o Encryption key: Default AWS Managed Key
5. Click Next.



Step 5: Add Source, Build, and Deploy Stages

1. Source Stage:
    - o Source provider: GitHub (via OAuth app)

o Select Repository (vishal003/Car-pwa-deploy-on-aws) and Branch (master).

2. Build Stage:

o Build provider: AWS CodeBuild

o Project name: Select pwa-codebuild-vb

o Build type: Single build

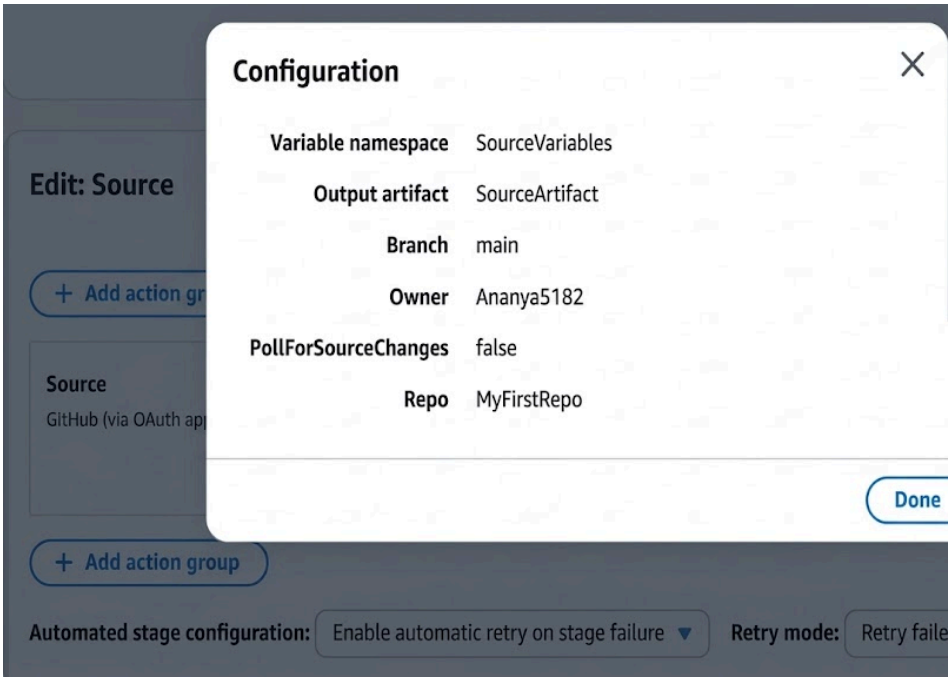
3. Deploy Stage:

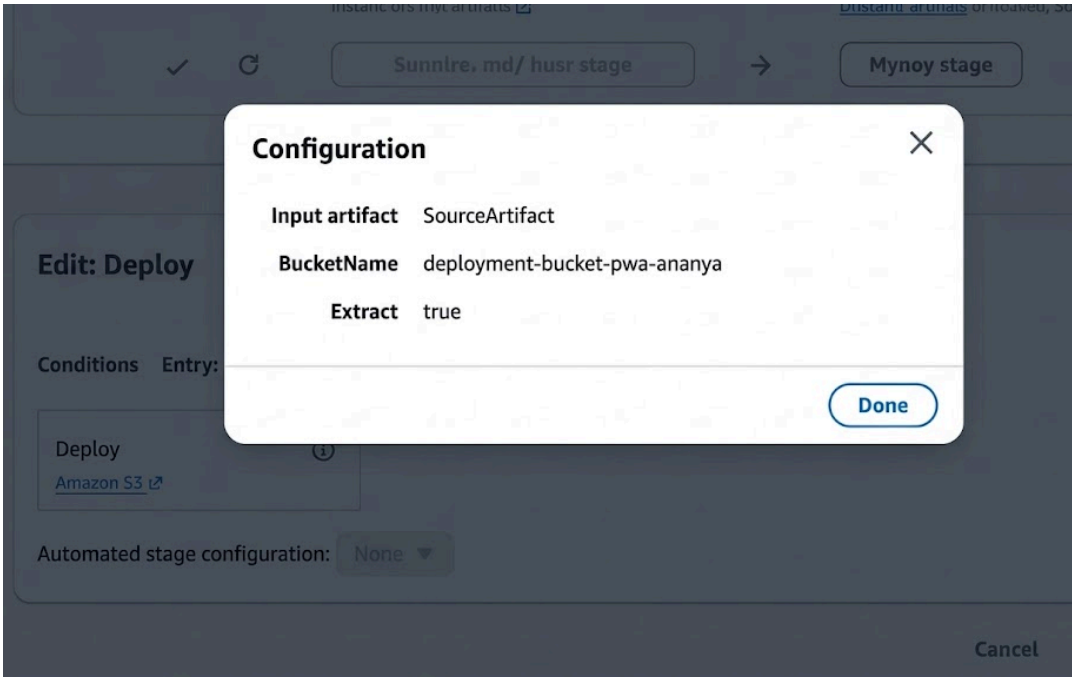
o Deploy provider: Amazon S3

o Bucket: Select deployment bucket (deployment-bucket-pwa-vb)

o Check the Extract file before deploying.

4. Review all settings and click Create pipeline.

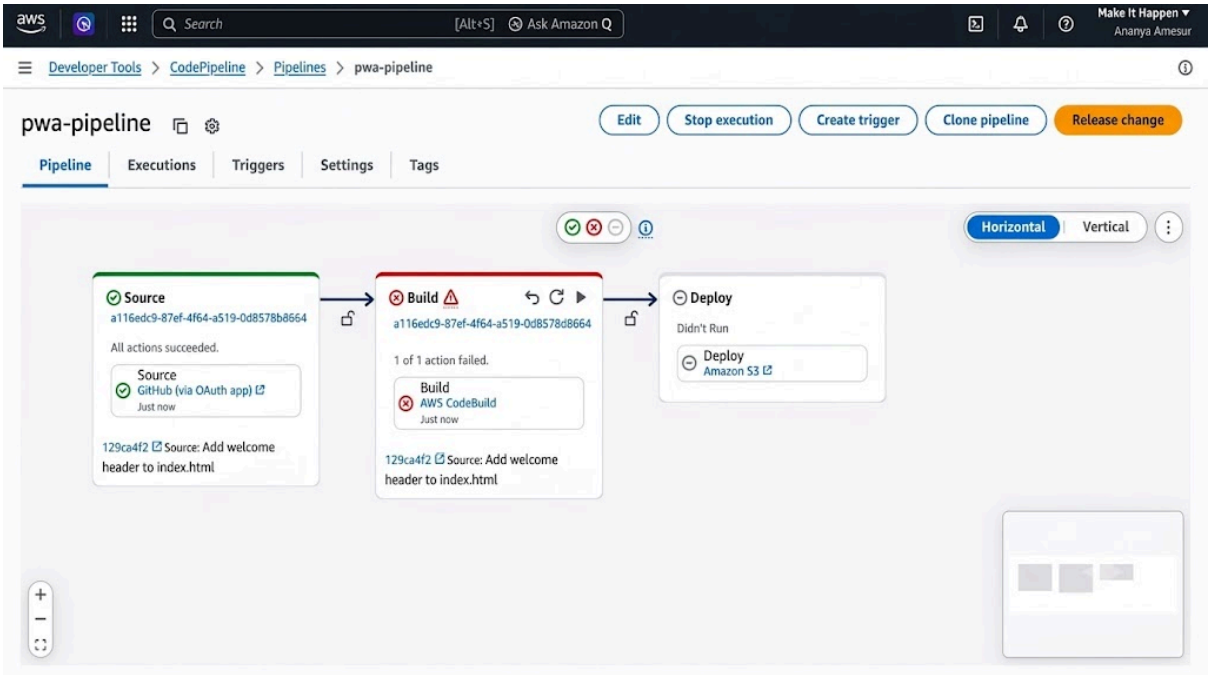




Phase 3: Troubleshooting IAM Permissions & Fixing Pipeline Build Failure

Step 6: Identify Build Failure

1. Upon pipeline creation, the initial automatic run fails during the Build stage due to missing S3 access permissions for the CodeBuild service role.



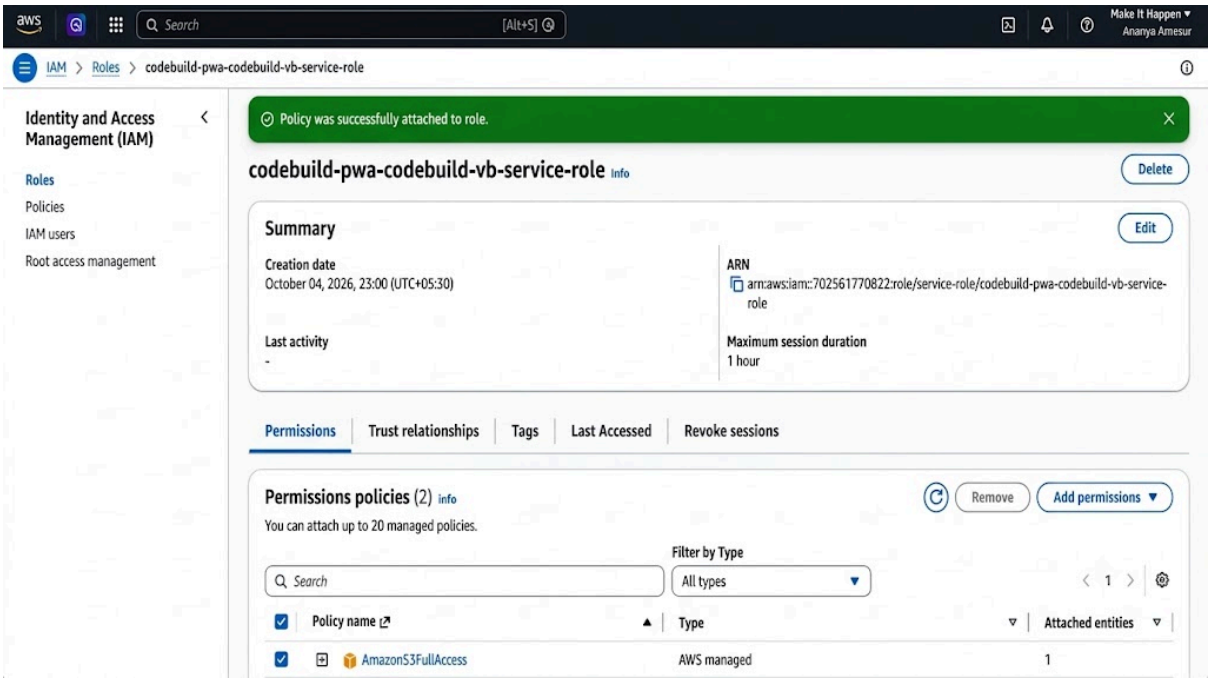
Step 7: Modify IAM Role Permissions

1. Navigate to **CodeBuild > Build projects > pwa-codebuild-vb**.

2. Go to **Build details**, locate the **Service role** ARN, and click it to open the AWS IAM Console in a new tab.

3. In the IAM Role (codebuild-pwa-codebuild-vb-service-role), click **Add permissions > Attach policies**.

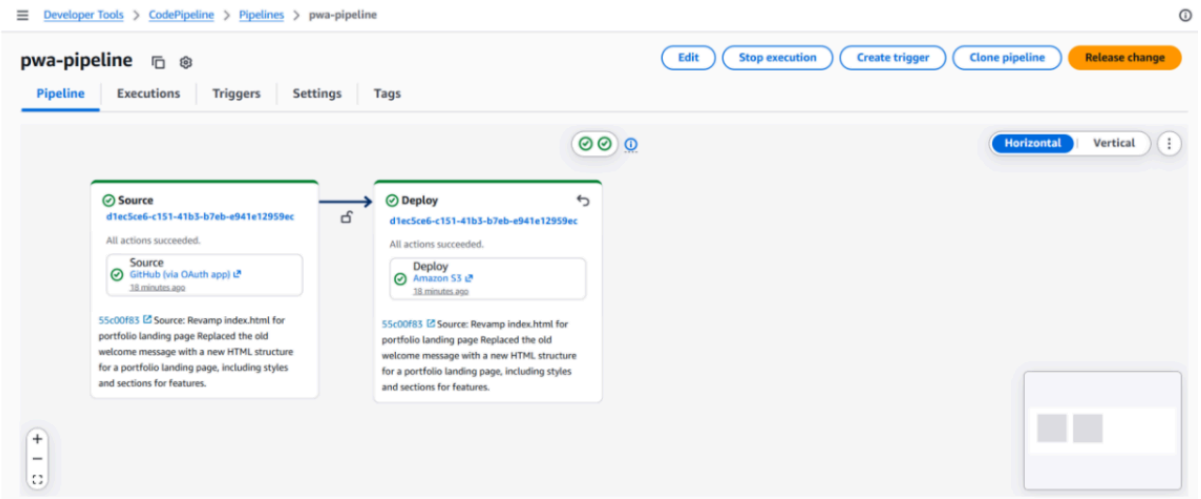
4. Search for AmazonS3FullAccess, select it, and click **Add permissions**.



Step 8: Retry Pipeline Execution

1. Return to CodePipeline > Pipelines > pwa-pipeline.

2. Verify that all two stages (Source and Deploy) [Build removed due to continuous error] complete successfully with green status indicators.



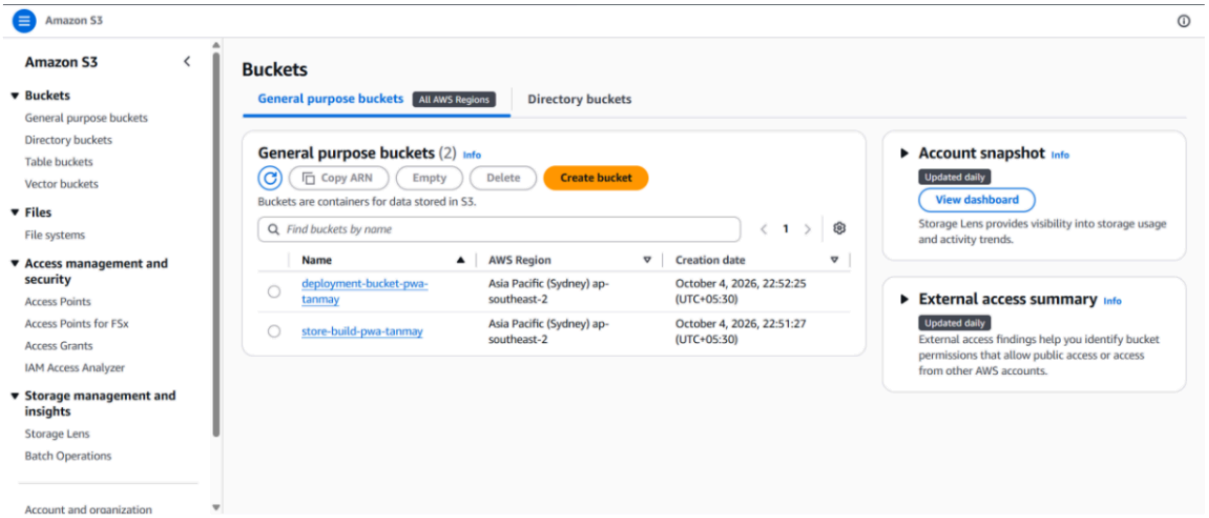
Phase 4: S3 Static Website Hosting & Public Access Policy

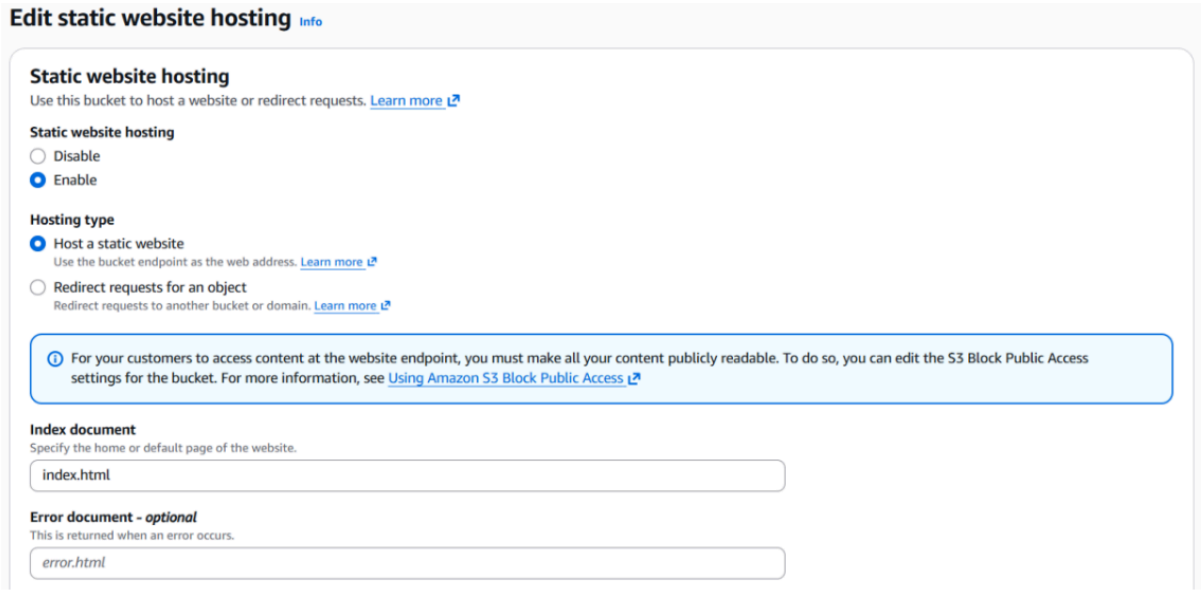
Step 9: Enable Static Website Hosting

1. Go to Amazon S3 > Buckets > deployment-bucket-pwa-vb.

2. Select the Properties tab and scroll to Static website hosting.

3. Click Edit, select Enable, choose Host a static website, and set Index document to index.html. Save changes.





Step 10: Configure Public Bucket Policy

1. Select the **Permissions** tab of deployment-bucket-pwa-vb.

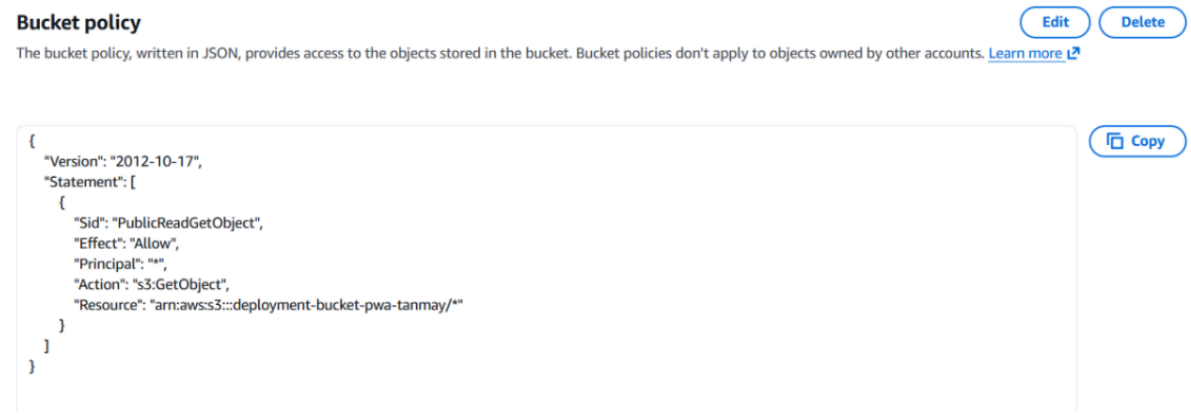
2. Disable **Block all public access** if enabled.

3. Scroll down to **Bucket Policy**, click **Edit**, and paste the following JSON policy:

JSON

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Sid": "PublicReadGetObject",
   "Effect": "Allow",
    "Principal": "*",
    "Action": [
```

```
        "s3:GetObject"
        ],
        "Resource": [
        "arn:aws:s3:::deployment-bucket-pwa-vb/*"
        ]
    }
]
}
```



Step 11: Verify Initial Live Website

1. Navigate back to Properties > Static website hosting on the S3 bucket.

2. Click the generated Bucket website endpoint URL.

3. Verify that the web application renders correctly in the browser.

Code Blame 184 lines (160 loc) · 4.57 KB Raw

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My Portfolio Landing Page</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
            line-height: 1.6;
            color: #333;
            background-color: #f9f9f9;
        }

        header {
            background-color: #1a1a1a;
            color: #fff;
            padding: 1.5rem 0;
            position: sticky;
            top: 0;
            z-index: 100;
        }

        .container {
            width: 90%;
            max-width: 1100px;
```

MyFirstRepo / index.html ↑ Top

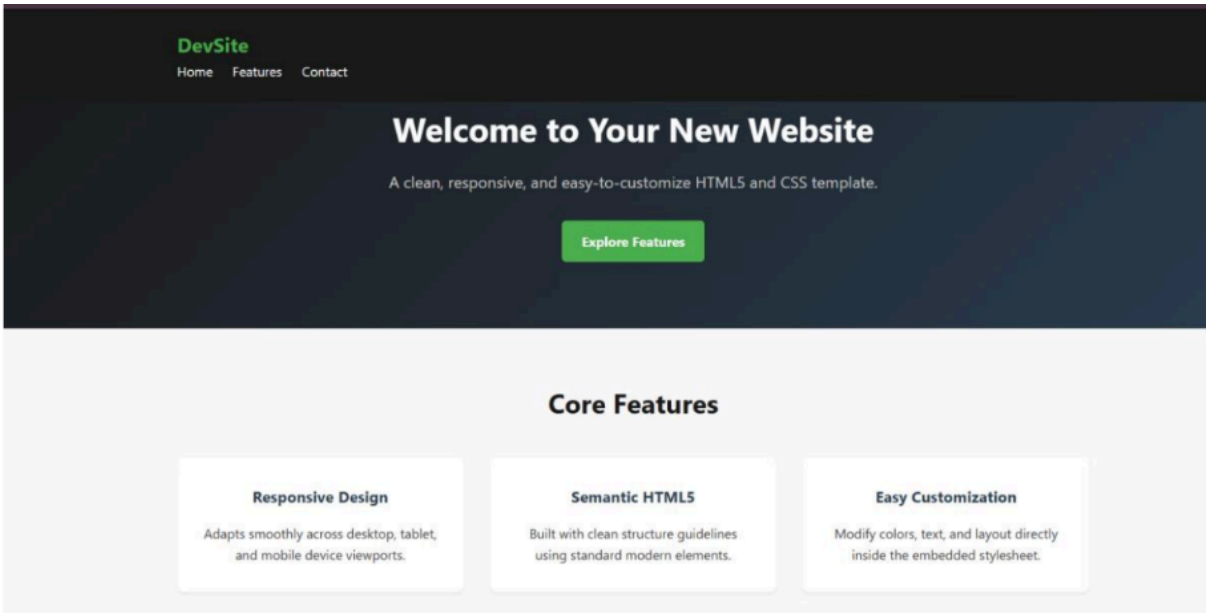
Code Blame 184 lines (160 loc) · 4.57 KB Raw

```

    <section id="features">
        <div class="container">
            <h2 class="section-title">Core Features</h2>
            <div class="features-grid">
                <div class="card">
                    <h3>Responsive Design</h3>
                    <p>Adapts smoothly across desktop, tablet, and mobile device viewports.</p>
                </div>
                <div class="card">
                    <h3>Semantic HTML5</h3>
                    <p>Built with clean structure guidelines using standard modern elements.</p>
                </div>
                <div class="card">
                    <h3>Easy Customization</h3>
                    <p>Modify colors, text, and layout directly inside the embedded stylesheet.</p>
                </div>
            </div>
        </div>
    </section>

    <footer id="contact">
        <div class="container">
            <p>&copy; 2026 DevSite. All rights reserved.</p>
        </div>
    </footer>

</body>
</html>
```



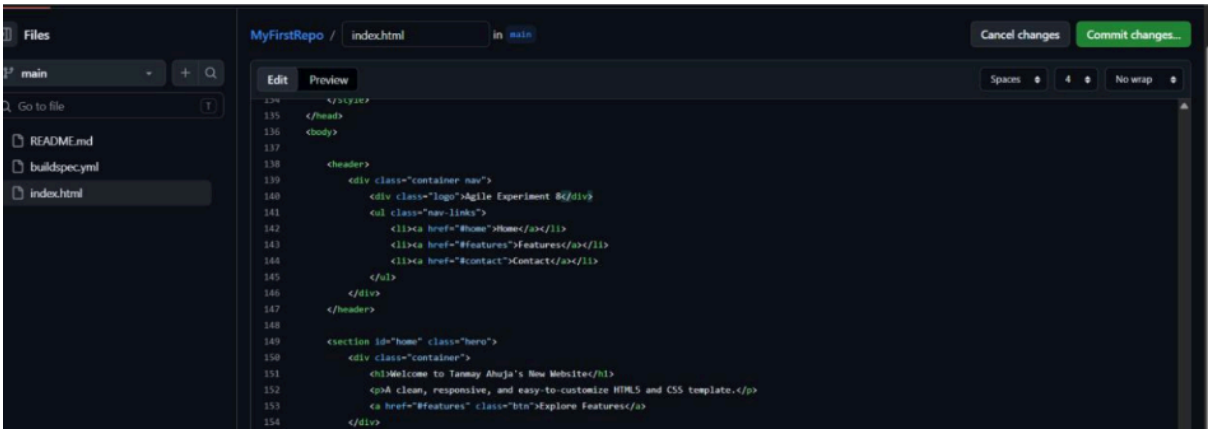
Phase 5: Testing Continuous Integration & Automated Redeployment

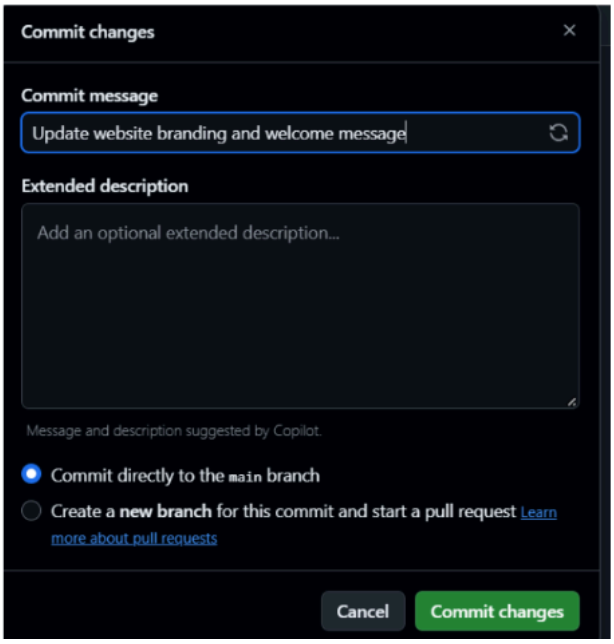
Step 12: Push Code Update to GitHub

1. Open the GitHub repository (vishal003/Car-pwa-deploy-on-aws) and edit index.html.

2. Modify banner text (e.g., update description text to include *"A.P. Shah Institute of Technology"*).

3. Commit and push changes directly to the master branch.



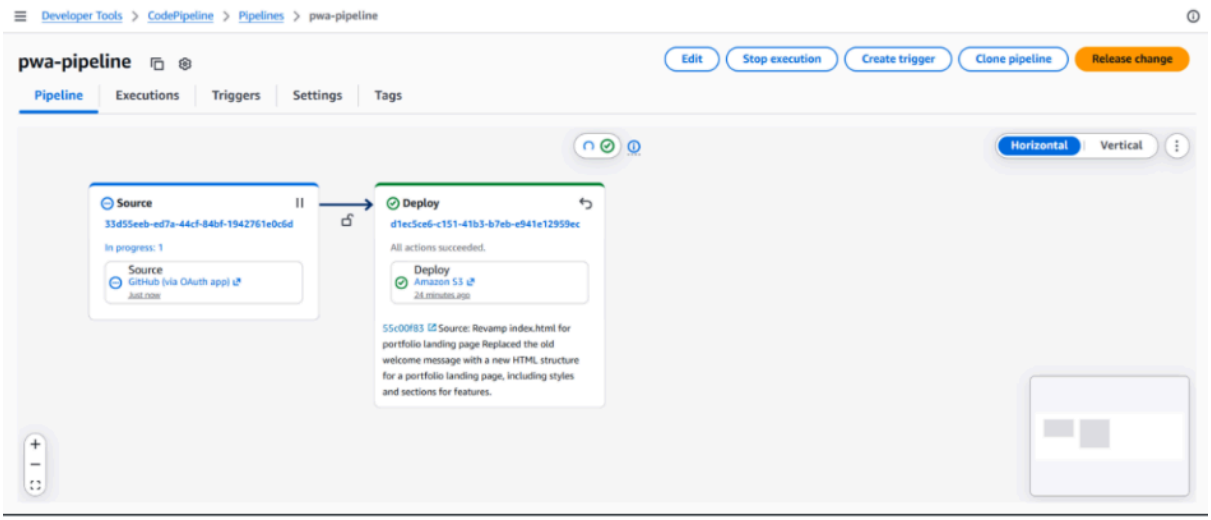


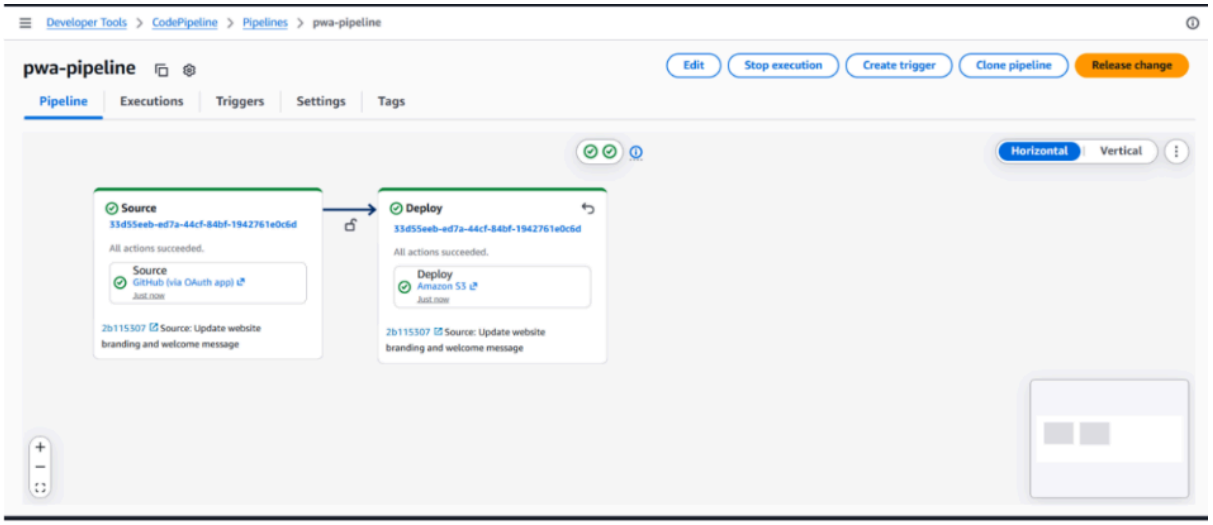
Step 13: Verify Auto-Trigger and Live Web Page Update

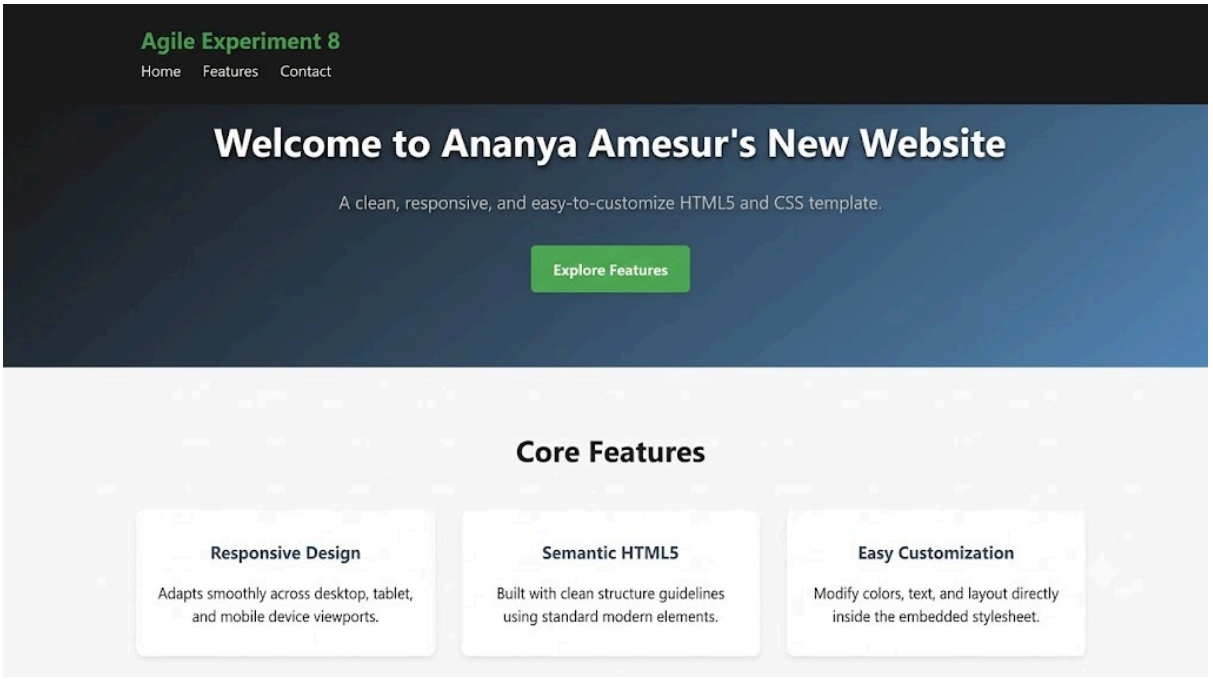
1. Open AWS CodePipeline. Observe that the pipeline automatically detects the commit (Update banner text...) and initiates execution (In progress).

2. Once the pipeline completes successfully, refresh the live S3 website endpoint URL.

3. Confirm that the webpage updates in real time with the new changes without any manual deployment steps.







Step 14: Resource Cleanup

1. Delete the created AWS CodePipeline (pwa-pipeline).

2. Delete the AWS CodeBuild project (pwa-codebuild-vb).

3. Empty and delete created Amazon S3 buckets (store-build-artifact-vb and deployment bucket-pwa-vb).

4. Delete associated IAM Roles.

Conclusion:

An automated end-to-end CI/CD pipeline was successfully configured and executed using GitHub, AWS CodePipeline, AWS CodeBuild, and Amazon S3 to deploy a responsive web application. Through this implementation, practical hands-on experience was gained in setting up source-driven automated release workflows, ephemeral build container environments, and automated artifact extraction into cloud storage. Furthermore, troubleshooting the initial build failure underscored the critical role of AWS IAM policies—specifically attaching the necessary S3 permissions to the CodeBuild service role to allow secure inter-service communication across AWS. Validating the pipeline with a live Git commit confirmed zero-downtime, fully automated redeployment, demonstrating how modern DevOps practices eliminate manual intervention and accelerate the software delivery lifecycle.